

UT3.- Gestión y manejo de ficheros

UT3.- Gestión y manejo de ficheros

Programación Orientada a Objetos

CE Desarrollo de aplicaciones en lenguaje Python



UT3: Gestión y manejo de ficheros



Elaboración propia (CC BY-NC-SA <<http://creativecommons.org/licenses/?lang=es>>)

1.- Introducción



Introducción

Python proporciona una variedad de herramientas y bibliotecas para trabajar con archivos y carpetas. En esta guía, exploraremos cómo utilizar estas herramientas para realizar tareas comunes como leer y escribir archivos, crear y eliminar carpetas, y más.

Archivos

Como sabemos los archivos permiten guardar información de forma persistente en nuestras computadoras. Los datos que forman un archivo tienen un formato concreto. Existen archivos de texto, de imágenes, de sonido y también los hay que guardan un programa ejecutable.

En esta unidad trataremos con archivos de **texto plano**, archivos **CSV** y archivos **JSON**. Leeremos datos de los mismos y crearemos archivos en estos formatos.

Rutas

En el tema que nos ocupa, una ruta es el camino hasta llegar a un archivo o directorio. Estas rutas pueden ser de dos tipos: **absolutas** y **relativas**.

- **Ruta absoluta.** Una ruta absoluta indica el camino para llegar a un archivo o directorio desde la raíz del sistema de archivos. Ejemplos:
 - c:\usuarios\pepe\archivo.txt.
 - /home/pepe/archivo.txt.
- **Ruta relativa.** Una ruta relativa indica el camino para llegar a un archivo o directorio tomando como punto de partida el directorio actual de trabajo. Es relativa con respecto al directorio actual de trabajo. **En el caso de Python será el directorio desde el que ejecutamos la aplicación.**
- La función **open()** lleva como primer argumento una ruta (path). Al intentar abrir un archivo mediante **open("archivo.txt", "r", encoding="utf-8")**, Python realiza una búsqueda relativa en el directorio actual (desde el que ejecutamos la aplicación).

- Si el archivo existe en esta carpeta creará un objeto que apunte a él, en caso contrario lanzará una excepción del tipo **FileNotFoundError**. Lo mismo sucede si se accede al archivo mediante `open("archivos/archivo.txt", "r", encoding="utf")`, Python buscará la carpeta archivos dentro del directorio actual.

En Python se puede trabajar tanto con **rutas absolutas como relativas**, siendo siempre conscientes de la operación que se está llevando a cabo.

En las aplicaciones que desarrollemos, utilizaremos el carácter **/ (forward slash)** para indicar directorios, independientemente del sistema operativo utilizado.

Python dispone de varios **módulos o librerías** que pueden ayudarnos a conocer el directorio actual.

- Por ejemplo, **el módulo os (import os)** proporciona un conjunto de funciones para interactuar con el sistema operativo en la gestión de archivos y directorios.

Entre estas funciones podemos encontrar las siguientes:

- **os.path.abspath(path)**. Retorna la ruta absoluta del path indicado.
- **os.path.dirname(path)**. Retorna el directorio asociado a la ruta indicada.
- **os.path.join(path, *paths)**. Retorna una ruta absoluta a partir de varias.
- **os.getcwd()**. Retorna la ruta o directorio actual de trabajo. Desde el que ejecuto mi aplicación.

Para conocer la ruta absoluta del archivo Python que se está ejecutando podemos hacer uso de las funciones **os.path.abspath()** y **os.path.dirname()** tal y como se muestra a continuación:

```
1  import os
2
3  print("Uso de las funciones os.path.abspath() y os.path.dirname()")
4  print("-----")
5
6  # Ruta absoluta del archivo Python que se está ejecutando
7  ruta_absoluta_archivo = os.path.abspath(__file__)
8  print(ruta_absoluta_archivo)
9
10 # Ruta absoluta del directorio donde se encuentra el archivo Python
11 ruta_absoluta_directorio = os.path.dirname(ruta_absoluta_archivo)
12 print(ruta_absoluta_directorio)
```

- Línea 7. La variable **__file__** guarda la ruta absoluta al archivo (o módulo) en el que se hace referencia a dicha variable. Mediante **os.path.abspath(__file__)** se obtiene

la ruta actual del archivo Python.

- Línea 11. Mediante `os.path.dirname(ruta)` se obtiene la ruta en la que se encuentra el archivo, sin incluir el nombre de este.
-

2.- Operaciones con archivos de texto plano



Archivos de texto plano

Los pasos que deben llevarse a cabo para trabajar con archivos sea cual sea su tipo son los siguientes:

- Abrir el archivo.
- Leer o escribir en el mismo.
- Cerrar el archivo.

La forma más sencilla y ágil de abrir archivos en Python se basa en la utilización de la **sentencia with**. Esta sentencia **cierra de forma automática** el archivo una vez que finaliza el bloque with, por tanto libera al programador de dicha tarea.

- En otro caso, la forma de cerrar un archivo es usando el método **archivo.close()**.

Formato de la sentencia:

```
1 | with open('archivo', 'modo apertura', encoding) as f:  
2 |     # operaciones
```

Los modos de apertura más comunes son los siguientes:

Opción de apertura	Utilidad
r (read)	Abrir en modo lectura. Opción por defecto.
w (write)	Abrir en modo escritura. Si el archivo existe sobrescribe en él.
a (append)	Abrir para añadir contenido al final del mismo.

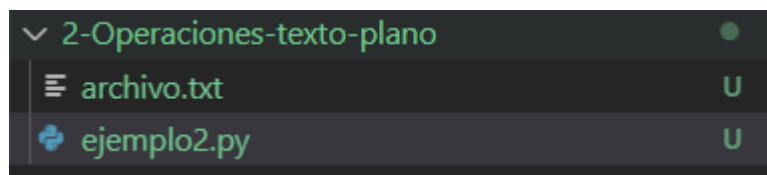
- La **codificación** es importante para interpretar correctamente los caracteres del mismo. Generalmente la codificación que usaremos será **"utf-8"**.
- Dentro del bloque **"with"** se detallan las operaciones que se realizarán con el archivo.

- Al abrir un archivo obtenemos un objeto de tipo “_io.TextIOWrapper” que proporciona acceso al archivo que tiene asociado.
- Los **métodos** que usaremos al trabajar con archivos son los siguientes. Son métodos asociados al objeto “_io.TextIOWrapper”.
 - **write()**. Para escribir en el archivo.
 - **read()**. Lee el contenido del archivo y lo retorna como str.
 - **readlines()**. Lee todas las líneas del archivo y las retorna como una lista (list).

Ejemplo 1. Cómo escribir en un archivo de texto

```
1  import os
2
3  # Generar un archivo de texto.
4
5  # Utilizamos una ruta absoluta para el archivo de texto.
6  directorio = os.path.dirname(os.path.abspath(__file__))
7  ruta_archivo_texto = os.path.join(directorio, "archivo.txt")
8
9  # Abrir el archivo en modo escritura
10 with open(ruta_archivo_texto, "w", encoding="utf-8") as archivo
11     # Escribir en el archivo
12     archivo.write("Todo hombre sabio teme tres cosas:\n")
13     archivo.write("la tormenta en el mar, \n")
14     archivo.write("la noche sin luna y \n")
15     archivo.write("la ira de un hombre amable.\n")
```

El resultado sería:



- Líneas 6 y 7. En este ejemplo, se utiliza una ruta absoluta para ubicar el archivo que se crea en la carpeta en la que está la propia aplicación python.
- Línea 10. Observa el modo de apertura del archivo, “w”. De esta forma, si el archivo no existe se crea, y si existe se sobrescribe.
- Línea 12 y siguientes. El método **write()** permite escribir en un archivo de texto.
- En la imagen se observa que el archivo “archivo.txt” se ha creado en la carpeta de la aplicación Python.

Ejemplo 2. Cómo leer un archivo de texto

```

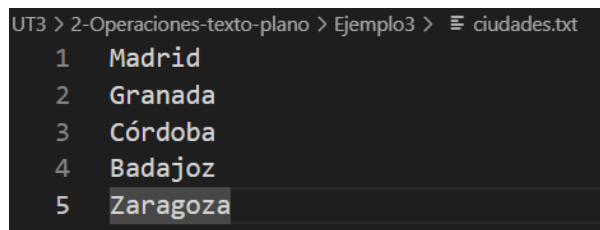
1  import os
2
3  # Leer de un archivo de texto.
4
5  # Utilizamos una ruta absoluta para el archivo de texto.
6  directorio = os.path.dirname(os.path.abspath(__file__))
7  ruta_archivo_texto = os.path.join(directorio, "archivo.txt")
8
9  # Abrir el archivo en modo lectura
10 with open(ruta_archivo_texto, "r", encoding="utf8") as archivo:
11     # Leer el contenido del archivo
12     contenido = archivo.read()
13     print(contenido)

```

- Líneas 5 y 6. Para este ejemplo, partimos del archivo generado en el ejemplo anterior y la aplicación leerá el contenido del mismo. Se utiliza una ruta absoluta para acceder a dicho archivo.
- Línea 10. El modo de apertura del archivo es “r” de lectura.
- Línea 12. El método read() lee todo el contenido del archivo en un objeto str

Ejemplo 3. Leer el contenido de un archivo de texto y guardar las líneas en una lista (list)

- En este caso, leeremos datos del archivo “ciudades.txt” y dejaremos el resultado en una lista.
- La imagen muestra el contenido de dicho archivo.



```

UT3 > 2-Operaciones-texto-plano > Ejemplo3 > ☰ ciudades.txt
1  Madrid
2  Granada
3  Córdoba
4  Badajoz
5  Zaragoza

```

Elaboración propia (CC BY-NC-SA
<http://creativecommons.org/licenses/?lang=es>)

```

import os

# Se utiliza una ruta absoluta para el archivo de texto.
directorio = os.path.dirname(os.path.abspath(__file__))
ruta_ciudades = os.path.join(directorio, "ciudades.txt")

```

```

6
7 with open(ruta_ciudades, "r", encoding="UTF8") as f:
8     contenido = f.readlines() # Leer el contenido del archivo y
9     contenido = [l.strip() for l in contenido] # Quitar '\n'
10    print(contenido) # Imprimir el contenido del archivo

```

- Líneas 4 y 5. Se busca el archivo en la carpeta que contiene la aplicación Python.
- Línea 7. El archivo se abre en modo lectura, “r”.
- Línea 8. El método `readlines()` sin argumentos lee todos los bytes del archivo en un objeto `list`. Es decir, cada elemento de la lista contiene al final el salto de línea (`\n`).
- Línea 9. El método “`strip()`” elimina los saltos de línea de cada cadena. En este caso se ha usado “list comprehensions” para crear la lista sin saltos de línea.

La salida de la aplicación es la que se muestra a continuación.

```
['Madrid', 'Granada', 'Córdoba', 'Badajoz', 'Zaragoza']
```

Elaboración propia (CC BY-NC-SA
<http://creativecommons.org/licenses/?lang=es>)

Ejemplo 4. Añadir líneas a un archivo de texto existente

En el ejemplo que sigue se añaden líneas al archivo “ciudades.txt”.

```

1 # Añadir líneas a un archivo
2 import os
3
4 # Se utiliza una ruta absoluta para el archivo de texto.
5 directorio = os.path.dirname(os.path.abspath(__file__))
6 ruta_ciudades = os.path.join(directorio, "ciudades.txt")
7
8 with open(ruta_ciudades, "a", encoding="UTF8") as f: # Abre el
9     f.write("Vigo\n") # Añadir una línea al archivo
10    f.write("La Coruña\n") # Añadir otra línea al archivo

```


Puede darse el caso de querer llevar a cabo una lectura línea a línea de un archivo y aplicar algún tratamiento a cada una de estas líneas de forma individual. Para recorrer un archivo por líneas basta con utilizar un bucle “for” con el objeto que apunta al archivo que proporciona “with”.

```
1 | with open("archivo.txt", "r", encoding="utf-8") as archivo:
2 |     for linea in archivo:
3 |         # procesamiento de la línea
```

En el ejemplo que se estudia a continuación se leerá un archivo de texto línea a línea, y por cada una de ellas se contará el número de vocales que contiene.

```
1 | import os
2 |
3 | # Leer línea a línea un archivo de texto
4 |
5 | # Ruta absoluta del archivo
6 | directorio = os.path.dirname(os.path.abspath(__file__))
7 | ruta_archivo = os.path.join(directorio, "archivo.txt")
8 |
9 | # Abrir el archivo en modo lectura
10 | with open(ruta_archivo, "r", encoding="utf-8") as archivo:
11 |     # Leer línea a línea
12 |     for linea in archivo:
13 |         numero_vocales = len([x for x in linea.lower()
14 |                               if x in "aeiouáéíóú"])
15 |         # Contar vocales en la línea y guardar en variable
16 |         print(f'{linea.strip()} tiene {numero_vocales} vocales')
```

- Línea 10. Apertura del archivo.
- Línea 12. Bucle “for” para recorrer el objeto archivo línea a línea.
- Línea 13. Contar las vocales de cada línea. Se crea una lista con las vocales y se obtiene la longitud de esa lista.
- Línea 15. Mostrar los resultados.

La salida de la aplicación será la que se observa en la imagen adjunta.

```
'Todo hombre sabio teme tres cosas:' tiene 12 vocales  
'la tormenta en el mar,' tiene 7 vocales  
'la noche sin luna y' tiene 6 vocales  
'la ira de un hombre amable.' tiene 10 vocales  
PS C:\Users\Sergio\Desktop\CLASES\5100-P00>
```

Elaboración propia (CC BY-NC-SA
<<http://creativecommons.org/licenses/?lang=es>>)

Tarea 1

3.- Operaciones con archivos CSV



Archivos CSV

Un archivo con extensión **CSV** es un archivo de texto en el que cada línea es un registro formado por uno o más campos separados por coma, de ahí su nombre, **CSV** o *comma separated values*.

- Son muy útiles para exportar datos entre aplicaciones como bases de datos, hojas de cálculo, etc. y además pueden ser tratados por las herramientas de análisis de datos de Python como Pandas.
- La librería **CSV** de Python proporciona utilidades para trabajar con este tipo de archivos. Por ello al principio de cada proyecto será necesario realizar la importación mediante **import CSV**.
- La forma de trabajar será la misma que se vio en el punto anterior, es decir, abrir el archivo, procesar la información y cerrar el archivo.

Para crear/escribir en un archivo CSV se necesitará un objeto **_CSV.writer**, este objeto dispone del método **writerow()** que escribe información en el archivo.

Para leer será necesario un objeto **_CSV.reader** que contiene el método **reader()** que será el encargado de la lectura.

Ejemplo 1. Generar un archivo CSV

En este caso partimos de una lista con nombre, apellido y edad de una serie de personas que serán guardadas en un archivo CSV.

```
# Crear un archivo CSV
import csv
import os

# Datos a escribir
datos = [
    ['Nombre', 'Apellido', 'Edad'],
    ['Juan', 'Pérez', 25],
    ['Ana', 'Gómez', 30],
    ['Pedro', 'García', 45],
    ['María', 'López', 35]
]
```

```

15 # Ruta absoluta del archivo
16 directorio = os.path.dirname(os.path.abspath(__file__))
17 ruta_archivo = os.path.join(directorio, 'datos.csv')
18
19 # Apertura del archivo en modo escritura
20 # newline='' en open() para evitar líneas en blanco adicionales
21 with open(ruta_archivo, mode='w', encoding="utf8", newline='') :
22     # Crear un objeto writer.
23     csv_writer = csv.writer(archivo, delimiter=',', lineterminator=
24     for elemento in datos:
        csv_writer.writerow(elemento)

```

- Línea 20. Apertura del archivo para escritura.
- Línea 22. Obtener el objeto writer a partir del archivo. Indicamos el separador de columnas (delimiter) y el de líneas (lineterminator).
- Línea 23 y siguientes. Mediante un bucle for se recorre la lista y cada elemento se escribe en el archivo mediante el método **“writerow(elemento)”**.

Ten en cuenta que cada objeto elemento es una lista (o sublista).

Ejemplo 2. Leer un archivo CSV

Ahora se llevará a cabo el proceso contrario, es decir, se parte de un archivo CSV y su información se traslada a una lista (list).

```

import csv
import os

# Ruta absoluta del archivo
directorio = os.path.dirname(os.path.abspath(__file__))
ruta_archivo = os.path.join(directorio, 'datos.csv')

# Lectura del archivo CSV
with open(ruta_archivo, mode='r', encoding="utf8") as archivo:
    csv_reader = csv.reader(archivo, delimiter=',') # Crea un i

    """
    # Mostrar el contenido fila a fila, cabecera y contenido
    for fila in csv_reader:
        print(fila)
    """

```

```

17 cabeceras = next(csv_reader) # Guardar las cabeceras en una
18 print(cabeceras)
19
20 """
21 l = list(csv_reader) # Guardar el contenido del archivo en l
22
23 # Mostrar el contenido fila a fila
24 for fila in l:
25     print(fila)
26
27 """
28
29
30 # Mostrar el contenido fila a fila
31 for fila in csv_reader:
32     print(fila)
33
34

```

Línea 1. Importación del módulo CSV.

- Línea 9. Apertura del archivo.
- Línea 10. Leer el archivo mediante el objeto reader.
- Línea 17. Guardar las cabeceras en una lista

```

['Nombre', 'Apellido', 'Edad']
['Juan', 'Pérez', '25']
['Ana', 'Gómez', '30']
['Pedro', 'García', '45']
['María', 'López', '35']

```

Elaboración propia (CC BY-NC-SA
<http://creativecommons.org/licenses/?lang=es>)

Ejemplo 3. Tratar las líneas de un CSV como diccionarios

La clase **csv.DictReader** permite manipular cada línea de un archivo CSV como un diccionario. Este diccionario tiene como claves las cabeceras del **archivo CSV** y como valor los datos propiamente dichos.

```

import csv
import os

```

```
3
4
5 try:
6     # Ruta absoluta del archivo
7     directorio = os.path.dirname(os.path.abspath(__file__))
8     ruta_archivo = os.path.join(directorio, 'departaments.csv')
9
10    # Abrir archivo en modo lectura
11    with open(ruta_archivo, "r", encoding="utf8") as archivo:
12        lector = csv.DictReader(archivo)
13
14        # Mostrar las cabeceras detectadas
15        print("Cabeceras:", lector.fieldnames)
16        print("-" * 40)
17
18        # Mostrar fila por fila
19        for linea in lector:
20            for k, v in linea.items(): # k = clave, v = valor, i
21                print(f"{k}: {v}", end=" ")
22            print() # salto de línea
23
24 except FileNotFoundError:
25     print("El archivo no existe")
```

La salida sería:



```
Cabeceras: ['ID', 'Nombre', 'Ubicacion']
-----
ID: 1  Nombre: Marketing  Ubicacion: Madrid
ID: 2  Nombre: Ventas  Ubicacion: Barcelona
ID: 3  Nombre: RRHH  Ubicacion: Valencia
```

Elaboración propia (CC BY-NC-SA
<<http://creativecommons.org/licenses/?lang=es>>)

Tarea 2

Tarea 2

Realizar ejercicio 1,2,3

4.- Tratamiento de archivos JSON



Archivos JSON

Aunque estos archivos nacieron unidos a Javascript, **JSON** viene de “**Javascript Object Notation**”, se ha convertido en el formato de archivo más utilizado en la distribución e intercambio de datos.

Un archivo **JSON** es un archivo de texto que se ajusta a una determinada estructura, y esta estructura es la misma que se utiliza en los diccionarios de Python. Es decir, un archivo JSON está formado por parejas clave-valor.

Para trabajar con archivos JSON en Python debemos importar el módulo JSON mediante **import JSON**.

En esta parte de la unidad veremos cómo convertir un diccionario de Python en una cadena con formato JSON y también el proceso contrario. Además crearemos y leeremos archivos JSON.

La tabla siguiente muestra la correspondencia entre objetos de Python y su equivalente en JSON.

Python	JSON
dict	Object
list, tuple	Array
str	String
int, float	Number
True, False	true, false
None	null

Las funciones del módulo JSON que estudiaremos son las siguientes:

- **JSON.load()**. Retorna un diccionario a partir de una cadena JSON. La cadena puede ser tomada de un archivo mediante **JSON.load(archivo)**.
- **JSON.dump()**. Escribe un diccionario Python en un archivo JSON.

- **JSON.dumps()**. Convierte un diccionario Python en cadena JSON.

Ejemplo 1. Convertir un diccionario Python en archivo JSON

En el ejemplo que sigue, el objeto datos (dict) se escribe en el archivo **datos.JSON**.

```
1  # Convertir diccionario a JSON
2  import json
3  import os
4  from datetime import date
5
6  # Creamos el Diccionario
7  alumno = {
8      "nombre": "Ana",
9      "apellidos": "Rico Casa",
10     "fecha_nacimiento": date(1990, 5, 15).isoformat(), # Formato
11     "notas": {
12         "Python": 8,
13         "Django": 7,
14         "Flask": 9
15     }
16 }
17
18 # Ruta absoluta del archivo
19 directorio = os.path.dirname(os.path.abspath(__file__))
20 ruta_archivo = os.path.join(directorio, 'datos_alumno.json')
21
22 # Guardar diccionario como JSON, Mediante el parámetro indent=4.
23 with open(ruta_archivo, 'w', encoding="utf-8") as archivo:
24     json.dump(alumno, archivo, indent=4, ensure_ascii=False) #
25
26 print(f"Archivo JSON creado en: {ruta_archivo}")
```

El JSON resultante se verá así:

```
Operaciones-archivos-JSON > {} datos_alumno.json > ...
{
  "nombre": "Ana",
  "apellidos": "Rico Casa",
  "fecha_nacimiento": "1990-05-15",
  "notas": {
    "Python": 8,
    "Django": 7,
    "Flask": 9
  }
}
```

Elaboración propia (CC BY-NC-SA
<<http://creativecommons.org/licenses/?lang=es>>)

Ejemplo 2. Cargar un archivo JSON en un diccionario Python

En este ejemplo leeremos el contenido de un archivo JSON y lo convertiremos en un diccionario de Python.

```
1  # De JSON a diccionario
2  import json
3  import os
4
5  # Ruta absoluta del archivo
6  directorio = os.path.dirname(os.path.abspath(__file__))
7  ruta_archivo = os.path.join(directorio, 'datos_alumno.json')
8
9  # Leer JSON y convertirlo a diccionario
10 with open(ruta_archivo, 'r', encoding="utf-8") as archivo:
11     datos_alumno = json.load(archivo) # Usa json.load() para coi
12
13 print(datos_alumno)
```

Tarea 2

Tarea 2

Ejercicios 4, 5 y 6

5.- Excepciones en el tratamiento de archivos



Excepciones en el tratamiento de archivos.

El intérprete de Python lanza una excepción cuando en la ejecución de un programa se produce algún error. Por ejemplo, abrir un archivo que no existe, escribir en un archivo para el que no se tienen permisos, error al conectar a un servidor de bases de datos, etc.

Las excepciones más comunes que pueden darse al trabajar con archivos son:

- **FileNotFoundError**. Esta excepción se lanza cuando se intenta acceder a un archivo que no existe.
- **PermissionError**. Cuando un programa intenta llevar a cabo una operación para la que no se tiene permiso.
- **IOError**. Esta excepción engloba un conjunto más amplio de operaciones, tales como intentar leer de un archivo que está abierto para escritura, acceder a un archivo que no existe o para el que no se tienen permisos, etc.

Como sabemos las excepciones en Python se tratan mediante el bloque **try-except**. Si al ejecutar el código del bloque **try** se produce una excepción, se ejecutan las instrucciones del **bloque except**. Si no se lanza la excepción/es controladas en este bloque **except** se ejecuta el código asociado al bloque **else**. El código asociado a **finally** se ejecuta siempre.

Los bloques **else** y **finally** son opcionales.

En el ejemplo que sigue, se muestra el funcionamiento de **try-except**. Se intenta escribir en un archivo para el que no se tienen permisos de escritura. La aplicación termina con normalidad mostrando el mensaje asociado a la excepción.

```
1  # Manejo de archivos con try - except - else - finally
2
3  try:
4      # Intentar abrir un archivo en modo lectura
5      with open("ejemplo.txt", "r", encoding="utf-8") as archivo:
6          contenido = archivo.read()
7
8  except FileNotFoundError:
9      print("Error: El archivo no existe.")
10
```

```
11 else:
12     # Se ejecuta solo si NO hubo excepción
13     print("Contenido del archivo:")
14     print(contenido)
15
16 finally:
17     # Se ejecuta siempre
18     print("Programa finalizado.")
```

En el ejemplo que sigue, se muestra el funcionamiento de **try-except**. Se intenta escribir en un archivo para el que no se tienen permisos de escritura. La aplicación termina con normalidad mostrando el mensaje asociado a la excepción.

```
1 # Manejo de excepción PermissionError
2
3 try:
4     # Intentar escribir en un archivo protegido (ejemplo en raíz del
5     with open("/protegido.txt", "w", encoding="utf-8") as archivo:
6         archivo.write("Intentando escribir en un archivo protegido..")
7
8 except PermissionError:
9     print("Error: No tienes permisos para escribir en este archivo.")
10
11 else:
12     print("Archivo escrito correctamente.")
13
14 finally:
15     print("Programa finalizado.")
```

6.- Operaciones con ficheros y directorios

Operaciones con ficheros

En este apartado vamos a mostrar como podemos realizar en Python las operaciones sobre ficheros de: copia, mover, eliminar y comprobar si existe un fichero

Copiar un fichero

```
1  import os                # Importar la libreria de archivos y cal
2  import shutil            # Importar la libreria de archivos y cal
3
4  # Copiar un fichero
5
6  # Ruta absoluta del archivo
7  directorio = os.path.dirname(os.path.abspath(__file__))
8  ruta_archivo = os.path.join(directorio, "fichero_texto.txt")
9  ruta_archivo_copia = os.path.join(directorio, "fichero_texto_cop
10
11  try:
12      # Intentar copiar un fichero
13      shutil.copyfile(ruta_archivo, ruta_archivo_copia)
14
15  except FileNotFoundError:
16      print("Error: El archivo no existe.")
17
18  else:
19      # Se ejecuta solo si NO hubo excepción
20      print("Fichero copiado correctamente.")
21
22  finally:
23      # Se ejecuta siempre
24      print("Programa finalizado.")
```

Mover un fichero

```
import os                # Importar la libreria de archivos y cal
import shutil            # Importar la libreria de archivos y cal
```

```
3
4
5 # Mover un fichero
6 directorio = os.path.dirname(os.path.abspath(__file__))
7 ruta_archivo = os.path.join(directorio, "fichero_texto.txt")
8 ruta_archivo_copia = os.path.join(directorio, "fichero_texto_copia.txt")
9
10 try:
11     # Intentar mover un fichero
12     shutil.move(ruta_archivo, ruta_archivo_copia)
13 except FileNotFoundError:
14     print("Error: El archivo no existe.")
15
16 else:
17     # Se ejecuta solo si NO hubo excepción
18     print("Fichero movido correctamente.")
19
20 finally:
21     # Se ejecuta siempre
22     print("Programa finalizado.")
```

Eliminar un fichero

```
import os                # Importar la libreria de archivos y carpetas
import shutil            # Importar la libreria de archivos y carpetas

# Eliminar un fichero
directorio = os.path.dirname(os.path.abspath(__file__))
ruta_archivo = os.path.join(directorio, "fichero_texto_copia.txt")

try:
    # Intentar mover un fichero
    os.remove(ruta_archivo)

except FileNotFoundError:
    print("Error: El archivo no existe.")

else:
    # Se ejecuta solo si NO hubo excepción
    print("Fichero borrado correctamente.")

finally:
```



```
# Se ejecuta siempre  
print("Programa finalizado.")
```

Comprobar si existe un fichero

```
1  import os                # Importar la libreria de archivos y cai  
2  import shutil            # Importar la libreria de archivos y cai  
3  
4  # Comprobar si existe un fichero  
5  
6  directorio = os.path.dirname(os.path.abspath(__file__))  
7  ruta_archivo = os.path.join(directorio, "fichero_texto.txt")  
8  
9  if os.path.exists(ruta_archivo):  
10     print("El fichero existe")  
11 else:  
12     print("El fichero no existe")
```

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0**
<<http://creativecommons.org/licenses/by-sa/4.0/>>